

From Vector Add to Matrix Add

- Known pattern: one CUDA thread computes one output element.
- New idea: the output element is identified by (row, col), not only by i.
- Keep the same ownership rule: one thread owns one matrix entry.

```
C[row * cols + col] = A[row * cols + col]  
                    + B[row * cols + col];
```

Hint: Start from the vector-add mental model, then replace the single index with row and column coordinates.

2D Thread Coordinates

Use x for columns and y for rows.

Column

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
```

Row

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
```

- `blockIdx.x / threadIdx.x` move horizontally through columns.
- `blockIdx.y / threadIdx.y` move vertically through rows.

Row-Major Addressing

- The matrix is still stored as one flat array in device memory.
- Row-major means complete rows are stored next to each other.

```
int i = row * cols + col;
```

- Skip row complete rows, each containing cols entries.
- Then move col positions into the current row.

Hint: For cols = 700, matrix entry (3, 5) is stored at index $3 * 700 + 5$.

Choosing a 2D Block and Grid

Suggested block size

```
dim3 threads(16, 16);
```

- This creates 256 threads per block, a common CUDA scale.

Grid covers all columns and rows

```
dim3 blocks((cols + 15) / 16,  
            (rows + 15) / 16);
```

Hint: The +15 rounds up, so partial blocks at the right and bottom edges are included.

Why the Bounds Check Is Required

- The rounded-up grid may create extra threads outside the matrix.
- Those threads must not read or write memory.
- Both row and col can be out of range, so the guard is two-dimensional.

```
if (row < rows && col < cols) {  
    ...  
}
```

Hint: This is the 2D version of the $i < n$ check from vector add.

Host-Code Checklist

- Allocate host arrays for A, B, and C.
- Set rows = 1000 * factor and cols = 700.
- Use entries * sizeof(float) for allocation sizes.
- Initialize A and B with nested row/col loops.
- Copy A and B to the GPU.
- Launch the 2D kernel with blocks and threads.
- Copy C back to the host.
- Free device and host memory.

Hint: Keep this as a checklist. Do not paste a full solution before you have the indexing correct.

Validation Hints

- Choose inputs where the expected answer is easy to compute.
- A useful pattern is $A[\text{row}, \text{col}] = \text{row}$ and $B[\text{row}, \text{col}] = \text{col}$.

```
C[row, col] = row + col;
```

Check at least three positions

- Top-left: (0, 0)
- Middle: (rows / 2, cols / 2)
- Bottom-right: (rows - 1, cols - 1)

Hint: Validating only one entry can miss row/column indexing mistakes.